Notes On a Reverse Engineering Algorithm

v0.1, 4 November 2026

1 Comments by the author

This problem has interested me for a long time, I created this document for my notes and formal proofs on different methods of navigating structure of the "space" of the compiler algorithm and the subject, now note that this is not a normal "paper" in the sense, it is not meant to be — these are raw ideas that came to me while solving this problem. Moving forward expect little to no filler nor explanation of the concepts.

2 The machine model

Choose a word size w and let $\mathbb{Z}_m = \mathbb{Z}/m\mathbb{Z}$. A register file is a finite set $\mathcal{R}$ of byte cells, while memory is a total function mapping addresses to bytes. The state space is

$$\Sigma \;=\; \mathbb{Z}_{2^8}^{\mathcal{R}} \times \mathbb{Z}_{2^8}^{\mathbb{Z}_{2^{64}}}.$$

A straight-line fragment P defines a function $[\![P]\!]$ from Σ into Σ, while an ABI defines an observation π from Σ into a finite set Y (for instance, the low byte of the return register together with final memory). The observable function for P is the composite $f_P = \pi \circ [\![P]\!]$.

All that follows is relative to this model. Faults, page permissions, timing, and concurrency are outside Σ, and none of the statements here addresses them.

3 Boxes

Definition 3.1. A *box* is a first-order formula $A(\sigma)$ in the theory of fixed-width bit-vectors and arrays, whose free variables range over Σ. Its extension is $\mathrm{Sat}(A) = \{\sigma \in \Sigma : \sigma \models A\}$.

A box is not a Cartesian product. For example, the formula $p = q$ over two pointer registers defines a box whose extension is the diagonal, not a rectangle. Boxes form a Boolean algebra under $\wedge$, $\vee$, $\neg$, and Sat is a homomorphism of Boolean algebras into the power set of Σ.

Proposition 3.2. *For boxes A and B,*

$$\mathrm{Sat}(A) \subseteq \mathrm{Sat}(B) \iff A \wedge \neg B \text{ is unsatisfiable}, \qquad \mathrm{Sat}(A) \cap \mathrm{Sat}(B) \neq \emptyset \iff A \wedge B \text{ is satisfiable}.$$

Proof. $\mathrm{Sat}(A) \setminus \mathrm{Sat}(B) = \mathrm{Sat}(A \wedge \neg B)$, and a set is empty exactly when its defining formula has no model. The second statement is $\mathrm{Sat}(A) \cap \mathrm{Sat}(B) = \mathrm{Sat}(A \wedge B)$. □

The solver returns either sat, unsat or unknown. An answer of sat includes a model, which provides a concrete witness state; unsat proves emptiness; unknown establishes nothing and is reported as such.

Example 3.3 (Aliasing)**.** Let P first store 7 at p, then 9 at q, and finally load the byte at p. When memory is represented by a single array M, the result is

$$\mathrm{read}(\mathrm{store}(\mathrm{store}(M,p,7),q,9),p) \;=\; \begin{cases} 9 & p = q, \\ 7 & p \neq q, \end{cases}$$

by the read-over-write axioms. Thus, for any allocation box B, the boxes $B \wedge (p = q)$ and $B \wedge (f_P = 9)$ have identical extensions. Proposition 3.2 establishes both inclusions using two unsatisfiability checks.

Proposition 3.4 (Projection of irrelevant inputs)**.** *Let $r \in \mathcal{R}$. Then f_P is independent of the cell r if and only if the formula*

$$f_P(\sigma) \neq f_P(\sigma[r := v])$$

in the free variables σ, v is unsatisfiable.

Proof. Independence of r means $f_P(\sigma) = f_P(\sigma')$ whenever σ and σ' agree off r. Every pair of this kind has the form $(\sigma, \sigma[r := v])$. The formula is satisfiable precisely when such a pair provides a counterexample. □

Only once this check succeeds is r removed from the input interface.

4 Execution fibers

Definition 4.1. Given f from X into Y and $y \in Y$, the *fiber* over y is $f^{-1}(y) = \{x \in X : f(x) = y\}$.

Example 4.2. Let $f(x) = 6x + 4$ on $\mathbb{Z}_{256}$. Then $f^{-1}(22) = \{3, 131\}$.

Proof. $6x + 4 \equiv 22 \pmod{256}$ is $6x \equiv 18 \pmod{256}$. Since $\gcd(6, 256) = 2$ divides 18, this is equivalent to $3x \equiv 9 \pmod{128}$. Because $3 \cdot 43 = 129 \equiv 1 \pmod{128}$, multiplying by 43 yields $x \equiv 387 \equiv 3 \pmod{128}$. The residues of $\mathbb{Z}_{256}$ that are congruent to 3 modulo 128 are 3 and 131. □

In general, $|f^{-1}(y)| \in \{0, \gcd(a, 2^w)\}$ for $f(x) = ax + c$ on $\mathbb{Z}_{2^w}$, so the observation by itself cannot distinguish the inputs when a is even.

Proposition 4.3 (Augmented observation)**.** *Let $x \in \{0, \ldots, 255\}$ and $\tilde{f}(x) = (y, k)$, where $6x+4 = y + 256k$ with $0 \leq y < 256$. Then $\tilde{f}$ is injective, and $x = (y + 256k - 4)/6$.*

Proof. The integer $6x + 4$ can be recovered from (y, k) as $y + 256k$, while x can be recovered from $6x + 4$ because multiplication by 6 is injective on $\mathbb{Z}$. □

For the fiber in Example 4.2: $6 \cdot 3 + 4 = 22$, $(22, 0)$ follows, while $6 \cdot 131 + 4 = 790 = 22 + 3 \cdot 256$ yields $(22, 3)$. The quotient k is additional observed state: the bits lost when truncating to one byte.

Remark 4.4 (Three coordinate systems). The same function can be represented in three ways: modular arithmetic on $\mathbb{Z}_{256}$; a Boolean circuit over $\mathbb{F}_2^8$ with explicit carry bits; and an embedding into $\mathbb{R}^8$ whose points are constrained to the hypercube vertices $\{0, 1\}^8$. Each model has the fiber $\{3, 131\}$ over 22. If the last model is relaxed to the solid cube, the fiber expands to include non-vertex points. Thus, the discreteness constraint belongs to the exact model and cannot be omitted.

5 Compiler fibers

Let S denote a set of source texts, θ a compiler profile (compiler, version, flags), and C_θ the map from S to byte strings obtained by compiling and extracting the relevant function or image.

Definition 5.1. For a target byte string b, the *compiler fiber* is $C_\theta^{-1}(b) \subseteq S$. A *partial right inverse* on D is a map I from D into S such that $C_\theta(I(b)) = b$ for every $b \in D$.

Strict source recovery means producing an element of $C_\theta^{-1}(b)$. We distinguish two notions:

$$\begin{aligned} \text{exact:} \quad & C_\theta(s) = b, \\ \text{semantic:} \quad & f_{C_\theta(s)} = f_b \text{ on the modelled domain.} \end{aligned}$$

Exact recovery implies semantic recovery. The converse does not hold, because different register allocations or instruction selections can compute the same function.

5.1 The affine family

Proposition 5.2. *Let $f(x, y) = ax + by + c$ on $\mathbb{Z}_{2^w}$. Then*

$$c = f(0,0), \qquad a = f(1,0) - c, \qquad b = f(0,1) - c,$$

and distinct triples $(a, b, c) \in \mathbb{Z}_{2^w}^3$ specify distinct functions.

Proof. Evaluate at the three basis points. The formulas recover (a, b, c) from f, so any two triples with the same f must coincide. □

Proposition 5.2 provides the inverse I. A candidate counts as exact only when recompiling $I(b)$ reproduces the function bytes; otherwise, it is reported as semantic.

Remark 5.3 (Non-uniqueness for polynomials). Uniqueness does not hold beyond degree one. For $w \geq 1$, the product $x(x-1)$ is always even, and therefore

$$2^{w-1}x(x-1) \equiv 0 \pmod{2^w} \quad \text{for all } x.$$

Thus, g and $g + 2^{w-1}(x^2 - x)$ define the same function on $\mathbb{Z}_{2^w}$ despite having different coefficients. Polynomial functions on $\mathbb{Z}_{2^w}$ form a proper quotient of the polynomial ring, so a recovered coefficient vector is merely a representative of its class.

5.2 Whole-program search

A source grammar has a finite set of holes $H = \{h_1, \ldots, h_n\}$, a finite choice set K_i with costs κ_i from K_i into $\mathbb{N}$ for each hole, and a finite set of profiles Θ with cost κ_0. An assignment selects one choice for each hole, and substitution is literal and applied at every occurrence. The candidate space is therefore

$$\Omega = \Theta \times K_1 \times \cdots \times K_n, \qquad \kappa(\theta, k_1, \ldots, k_n) = \kappa_0(\theta) + \sum_{i=1}^{n} \kappa_i(k_i).$$

Each $\omega \in \Omega$ specifies a complete project s_ω, and ω is accepted when $C_\theta(s_\omega) = b$ for every target.

Proposition 5.4. *Sort each coordinate by cost, then initialize a priority queue with the tuple of minimum choices. Repeatedly pop a minimum-cost tuple, emit it, and push each tuple formed by advancing one coordinate to its next choice, skipping any tuple already seen. This emits every element of Ω exactly once, in nondecreasing cost order.*

Proof. Advancing a coordinate cannot lower the cost, because the choices in each coordinate are sorted. Each tuple other than the initial one has a predecessor formed by moving back one coordinate, with cost no greater than its own. Induction on the sum of the coordinate indices shows that every tuple is pushed, since its predecessor is popped. The seen set prevents duplicates. Suppose a tuple u is popped after a tuple v with $\kappa(u) < \kappa(v)$. When v was popped, some ancestor u' of u along a chain of predecessors was in the queue, with $\kappa(u') \le \kappa(u) < \kappa(v)$, contradicting the minimality of v. □

Corollary 5.5. *The first accepted candidate has the minimum declared cost among all accepted candidates in Ω. If the budget expires after N candidates, then every rejected candidate whose cost is below the last emitted cost has been tested. No conclusion can be drawn about Ω beyond those N, nor about any sources outside Ω.*

The size $|\Omega| = |\Theta| \prod_i |K_i|$ grows exponentially in n. Ordering makes the search useful, but does not make it polynomial.

6 Color refinement

Let $G = (V, E, \ell)$ be a directed multigraph with vertex labels ℓ and edge slots $s(e)$ recording operand position; operands of commutative operations share a slot.

Definition 6.1. Set $\chi_0 = \ell$ and

$$\chi_{t+1}(v) = \Big(\chi_t(v),\ \{\!\{(s(e), \chi_t(u)) : e = (u, v)\}\!\},\ \{\!\{(s(e), \chi_t(u)) : e = (v, u)\}\!\}\Big),$$

where $\{\!\{\cdot\}\!\}$ is a multiset. Let $\mathcal{P}_t$ denote the partition of V into classes of χ_t.

Lemma 6.2. *$\mathcal{P}_{t+1}$ refines $\mathcal{P}_t$. If $\mathcal{P}_{t+1} = \mathcal{P}_t$, then $\mathcal{P}_{t+k} = \mathcal{P}_t$ for all k. Stabilization takes at most $|V|$ rounds.*

Proof. $\chi_{t+1}(v)$ includes $\chi_t(v)$ as its first component, so identical new colors entail identical old colors. If two rounds produce the same partition, then χ_{t+1} is a bijective relabelling of χ_t on the classes, and the next round encounters the same multisets up to that relabelling. Each strict refinement increases the number of classes, which is bounded by $|V|$. □

Colors are compared as exact tuples, preventing hash collisions from merging distinct classes.

Proposition 6.3. *If $G \cong G'$ under a label- and slot-preserving isomorphism, the stable color histograms of G and G', refined jointly, are equal. Consequently, different histograms prove non-isomorphism.*

Proof. For an isomorphism φ, $\chi_t(\varphi(v)) = \chi_t(v)$ holds for all t by induction on t, since it maps the in- and out-neighbourhoods bijectively while preserving slots. □

Example 6.4 (The converse fails)**.** Let C_6 be the six-cycle and $2K_3$ be two disjoint triangles, both with uniform labels. Each graph is 2-regular and has six vertices, so every vertex receives the same color in every round and the histograms match. Nevertheless, the graphs are not isomorphic.

Betti numbers of the underlying 1-complex distinguish them: β_0 counts the components, and $\beta_1 = |E| - |V| + \beta_0$ is the cycle rank:

$$(\beta_0, \beta_1)(C_6) = (1, 1), \qquad (\beta_0, \beta_1)(2K_3) = (2, 2).$$

Remark 6.5. Structural matching and semantic equality are independent properties. On $\mathbb{Z}_{2^w}$, $x \cdot 3$ and $(x \ll 1) + x$ represent equal functions despite having different graphs. Conversely, $x + y$ and $x \oplus y$ share the same untyped topology but differ at $x = y = 1$.

7 Counterexample-guided synthesis

Given a finite parameter domain $\mathcal{P}$, a finite state domain Σ_0, and a violation formula $V(p, \sigma)$, the goal is

$$\exists p \in \mathcal{P} \;\; \forall \sigma \in \Sigma_0 : \neg V(p, \sigma).$$

The procedure maintains a finite set $T \subseteq \Sigma_0$ of counterexamples. It searches for p satisfying $\bigwedge_{\tau \in T} \neg V(p, \tau)$; if none exists, the grammar is exhausted. Otherwise, it searches for σ satisfying $V(p, \sigma)$. If none exists, it accepts p; if σ exists, it adds σ to T.

Proposition 7.1. *Accepted parameters are correct, and the procedure halts within at most* $|\mathcal{P}|$ *iterations.*

Proof. Acceptance is equivalent to the unsatisfiability of $V(p, \sigma)$ over the whole of Σ_0. If σ is added for p, then $V(p, \sigma)$ holds, which means p violates the new constraint and cannot be proposed again. Each iteration eliminates at least one parameter. □

Exhaustion establishes only that no parameter in this grammar works. Solver unknown is reported as unresolved.

8 Loop contracts

Let a loop have a one-lap transition $T(\sigma, \sigma')$, extracted from the machine code, a guard g, an initial condition Init, an invariant candidate J, a postcondition Q, and a ranking function ρ into $\mathbb{N}$.

Theorem 8.1. *Suppose the following formulas hold:*

$$\begin{aligned}
&\mathrm{Init}(\sigma) \Rightarrow J(\sigma),\\
&J(\sigma) \wedge g(\sigma) \wedge T(\sigma, \sigma') \Rightarrow J(\sigma'),\\
&J(\sigma) \wedge \neg g(\sigma) \Rightarrow Q(\sigma),\\
&J(\sigma) \wedge g(\sigma) \wedge T(\sigma, \sigma') \Rightarrow \rho(\sigma') < \rho(\sigma).
\end{aligned}$$

Then every execution starting from Init *terminates in a state satisfying* Q*.*

Proof. Induction on the number of laps shows that J holds at every loop head. During any run, ρ decreases strictly in $\mathbb{N}$ at each lap, so the run has at most $\rho(\sigma_0)$ laps. At exit, $\neg g$ holds along with J, and therefore Q. □

Each implication is verified by checking that its negation is unsatisfiable. For a counter i that runs to a bound n, $\rho = n - i$. A constant ρ fails the fourth condition, producing a counterexample.

Remark 8.2. An XOR accumulator $a' = a \oplus k$ is not affine over $\mathbb{Z}_{2^w}$, but it is linear over $\mathbb{F}_2^w$. Thus, the invariant language depends on the chosen coordinates, as noted in the remark on three coordinate systems.

9 Prototype refinement as a fixed point

Let F denote the set of recovered functions. For each $g \in F$, let $\delta(g)$ be the prototype inferred by the decompiler from the body of g: the model, arity, and a storage triple $(\text{space}, \text{offset}, \text{size})$ with a type for each parameter and the return. For a direct call site c in a caller h with target g, let $\gamma(c)$ denote the prototype inferred from the h side of the call.

Definition 9.1. The evidence is *consistent* at c if $\gamma(c)$ and $\delta(g)$ match in model, variadicity, custom storage, arity, every storage triple, and every type. A disagreement is a *conflict*.

Conflicts indicate disagreements between independent inferences; they do not prove anything about the true ABI.

A call is *trusted* if its caller has no unresolved relocation reads, bad data, or unimplemented instructions. For a caller h, define the manifest

$$m(h) = \{(g, \delta(g)) : h \text{ calls } g \text{ directly},\ g \neq h,\ \delta(g) \text{ supported}\},$$

where supported means non-variadic, using standard register or stack storage and supported types. Let Φ denote the operator that re-decompiles every caller h using the hypotheses $m(h)$ and recomputes δ and γ.

The decompiler checks each hypothesis against the storage it reconstructs. If it rejects any hypothesis, or the result contains unresolved machine operations, the previous result for h is retained.

Proposition 9.2. *Let $\eta(h)$ be a hash of $m(h)$ combined with the backend identity, and record the last adopted value $\eta^*(h)$. If iteration stops because $\eta^*(h) = \eta(h)$ holds for every caller h, the current state is a fixed point of Φ in this sense: rerunning Φ would provide each caller with exactly the hypotheses used in its last decompilation.*

Proof. For a deterministic backend, decompiling h depends only on h, the backend, and $m(h)$. Assuming there are no hash collisions, equal hashes imply equal manifests, so Φ reproduces the current evidence. □

No lattice order on prototypes is claimed under which Φ is monotone, so convergence is not generally guaranteed. Termination is enforced by a round limit R, a time budget, and a rule permitting each pair (h, η) to be attempted at most once. The outcome is reported as one of: fixed point, unresolved (a round with no adoption), or budget exhausted.

Example 9.3. Let $\texttt{leaf}(x, y) = x + y$ take x in EDI and y in ESI, and suppose the caller sets only EDI = 7 before the call. In its first decompilation, the caller sees one argument. With $m(\texttt{caller}) = \{(\texttt{leaf}, \delta(\texttt{leaf}))\}$, the call becomes binary, and the forwarded ESI appears as a caller parameter. A hypothesis assigning the first argument to the wrong register is rejected because of the storage mismatch. On rerun, $\eta^* = \eta$ and no reanalysis occurs; corrupting η^* triggers exactly one reanalysis, which restores the same η.

10 Translation unit fibers

At high optimisation levels, inlining, cloning and interprocedural register allocation couple the functions in a translation unit. This section examines how much the compiler fiber still decomposes into per-function pieces.

10.1 Summary locality

Express the source space as a product $S = S_1 \times \cdots \times S_m$ with one slot for each function, grouping functions that are inlined into one another into a shared slot. Let $N(j)$ denote the call-graph neighbours of slot j. Each slot has a finite summary set A_j, which records facts such as purity, the noreturn property, clobbered registers, propagated constants and memory effects. Set $A = A_1 \times \cdots \times A_m$ and denote by $a_{N(j)}$ the restriction of $a \in A$ to the neighbours of j.

Definition 10.1 (Summary locality)**.** There are maps σ_j from $S_j \times \prod_{i \in N(j)} A_i$ to A_j and F_j from $S_j \times \prod_{i \in N(j)} A_i$ to byte strings such that, for every $s \in S$, the system

$$a_j = \sigma_j(s_j, a_{N(j)}), \qquad j = 1, \ldots, m,$$

has exactly one solution $a(s) \in A$, and the bytes of function j are $C(s)_j = F_j(s_j, a(s)_{N(j)})$.

Definition 10.2. For a target $b = (b_1, \ldots, b_m)$ and $a \in A$, define the *local fiber* of slot j by

$$L_j(a) = \{s_j \in S_j : \sigma_j(s_j, a_{N(j)}) = a_j \text{ and } F_j(s_j, a_{N(j)}) = b_j\}.$$

Theorem 10.3. *Under summary locality,*

$$C^{-1}(b) = \bigsqcup_{a \in A} \prod_{j=1}^{m} L_j(a).$$

Proof. Let $s \in C^{-1}(b)$ and set $a = a(s)$. Then a solves the system and $F_j(s_j, a_{N(j)}) = b_j$ for every j, hence $s_j \in L_j(a)$. Conversely, suppose $s_j \in L_j(a)$ for every j. The first condition in $L_j(a)$ implies that a solves the system for s, so uniqueness gives $a = a(s)$, and the second condition then yields $C(s)_j = b_j$. The union is disjoint since $a(s)$ is a function of s. □

Corollary 10.4. *For a fixed a, searching the product requires $\sum_j |\Omega_j|$ candidate checks rather than $\prod_j |\Omega_j|$, where Ω_j is the candidate set for slot j. A summary is* observable *if it is determined by b; examples include a clobber set read from the callee bytes or a constant-propagation clone exposed as a symbol. If all summaries are observable, only one a is possible, and the search is linear in m.*

Proposition 10.5. *Suppose some summaries are unobservable, the call graph has treewidth w, and $|A_j| \le \alpha$ for every j. Then deciding whether $C^{-1}(b)$ is nonempty and finding one of its elements take $O(m \cdot \alpha^{w+1} \cdot \max_j |\Omega_j|)$ local checks.*

Proof. Every constraint in the system and every byte condition involves only slot j and its neighbours. Thus, the constraints form a network over variables a_j whose primal graph is the call graph. Take a tree decomposition of width w. Elimination along this decomposition enumerates at most α^{w+1} assignments for each bag, and tests the slots in each assignment against at most $\max_j |\Omega_j|$ candidates. The decomposition has $O(m)$ bags. □

For fixed w and α, the bound is polynomial in the number of functions. It offers no improvement for $|\Omega_j|$, which remains exponential in the number of holes in a single function.

Remark 10.6 (Refutation test). Summary locality predicts that changing slot j while keeping a fixed leaves $C(s)_i$ unchanged for $i \neq j$. A change to function i can therefore be explained only if it occupies slot j, neighbours a changed function in the call graph, or has a changed inlining decision.

10.2 Global inlining

gcc's interprocedural inliner greedily processes call edges from a priority queue, e, ordered by badness β_e. Each edge is also subject to per-caller limits, represented by an indicator $\lambda_e \in \{0, 1\}$, and a unit budget

$$B(s) = (1 + p) \max(N(s), L), \qquad N(s) = \sum_j n_j(s_j),$$

where p is the inline-unit-growth parameter, L is large-unit-insns, and n_j is the size estimate for slot j. For a fixed processing order, an edge e with growth g_e is included in the decision set $D(s)$ exactly when

$$\lambda_e = 1 \quad \text{and} \quad N(s) + \sum_{f \in D(s),\, f \prec e} g_f + g_e \le B(s).$$

Proposition 10.7. *Extend summary locality so that σ_j and F_j may also depend on D, and assume that $D(s)$ is a function of s. Then*

$$C^{-1}(b) = \bigsqcup_{(a,D)} \Big\{ s \in \prod_{j=1}^{m} L_j(a, D) : D(s) = D \Big\}.$$

Proof. Theorem 10.3 applies by the same argument, with (a, D) replacing a. The only condition that does not decompose over slots is $D(s) = D$, which remains as a filter. □

For a fixed order, the filter is a set of linear inequalities over the integers n_j and g_e. It is conjectured, but not proved, that a dynamic program over the slots that tracks prefix sums can decide the filter in pseudo-polynomial time in B. Two issues remain open. The order compares β_e across all edges, allowing up to $|E|!$ possible orders, while gcc recalculates g_e after each inlining and B uses the original size. It is unknown whether the entire order must be fixed, or whether it suffices to fix, for each rejected edge, the set of edges preceding it.

10.3 Indicator functions and spectra

Represent a set of sources by its indicator function. Over $\mathbb{F}_2$, Theorem 10.3 becomes

$$\mathbf{1}_{C^{-1}(b)} = \sum_{a \in A} \prod_{j=1}^{m} \mathbf{1}_{L_j(a)},$$

This holds at every s where the system has a unique solution, and may fail otherwise. The $\mathbb{F}_2$ affine hull of a set gives parity invariants through Gaussian elimination, but is not the same as the real affine hull. For example, $X = \{000, 110, 101, 011\}$ is an $\mathbb{F}_2$ hyperplane but has full affine rank over $\mathbb{R}$.

Assign each slot a graph G_j on S_j, and take their Cartesian product $G = G_1 \,\square\, \ldots \,\square\, G_m$. Its Laplacian eigenvectors are tensor products of eigenvectors from the factors, with eigenvalues given by addition. For the hypercube $Q_n = K_2 \,\square\, \ldots \,\square\, K_2$, the Walsh characters χ_T have eigenvalue $2|T|$. Over $\mathbb{R}$, the spectral coefficient of $\mathbf{1}_{C^{-1}(b)}$ at $T = (T_1, \ldots, T_m)$ is $\sum_a \prod_j$ of the coefficient of $\mathbf{1}_{L_j(a)}$ at T_j, again whenever $a(s)$ is unique.

Proposition 10.8. *Let $\mathcal{L}$ be the Laplacian of a finite graph, with orthonormal eigenvectors v_k and eigenvalues λ_k. Let X be a vertex set and write $\mathbf{1}_X = \sum_k c_k v_k$. Then*

$$\sum_k \lambda_k c_k^2 = |\partial X|,$$

is the number of edges with exactly one endpoint in X. On Q_n, taking $\hat{f}(T) = 2^{-n}\sum_x f(x)(-1)^{T\cdot x}$, this becomes $|\partial X| = 2^{n+1}\sum_T |T|\,\hat{\mathbf{1}}_X(T)^2$.

Proof. $\mathbf{1}_X^\top \mathcal{L}\mathbf{1}_X = \sum_{\{x,y\}\in E}(\mathbf{1}_X(x) - \mathbf{1}_X(y))^2$, where each summand is 1 for a cut edge and 0 otherwise. On Q_n, the characters are orthogonal with $\|\chi_T\|^2 = 2^n$, so $c_T^2 = 2^n\hat{\mathbf{1}}_X(T)^2$ and $\lambda_T = 2|T|$. □

10.4 Slot by slot search

First, read the relocations and call targets. These determine the call graph and all observable summaries before the search begins. Then restrict the slots one by one to their local fibers, following an elimination order along the call graph. Each step checks an exact equality, so no point in the fiber is discarded; the cost depends on the width of the order. This is sound only if the summaries fixed in advance are genuinely functions of b. Uniqueness of $a(s)$ is the sole hypothesis that neither reading the references first nor ordering the slots can provide.

10.5 Locality on real gcc

The refutation test used gcc 15.2 at -O3 on two zstd 1.5.6 translation units. Compiling the preprocessed units reproduces the shipped object code for 20 of 20 functions in huf_compress and 147 of 147 in zstd_compress. In turn, a probe—a single store to a static volatile variable—was inserted at the start of each definition, and every function was compared with relocations masked.

- huf_compress: 125 changes in the probed slot, 11 at neighbours, 0 from inlining and 4 unexplained. The 4 lie two calls away from the probe, across a function whose bytes do not change, and disappear under -fno-ipa-modref. A memory effect summary therefore passes through a function without changing its bytes. This is consistent with summary locality, but the summary is not observable.
- zstd_compress: 1146 changes in the probed slot, 174 at neighbours, 296 from inlining, 22 unexplained and 31 added or removed symbols, mostly part and cold splits. Of the inlining changes, 288 come from one call site in ZSTD_CCtxParams_setParameter that sits on an inlining threshold and flips with almost any size change elsewhere.

Global coupling does occur, but it is concentrated in a few marginal call sites, leaving D with few free bits in practice. Because a probe deliberately introduces a side effect, these counts may exaggerate the impact of an edit that preserves the summaries.

11 Thermodynamics of destroyed information

Compilation discards information. The sources

```
int f(void) { return 42; }
```
and
```
int f(void) { int x = 999; return 42; }
```

compile to identical bytes and therefore belong to the same fiber; nothing computed from those bytes can identify which source was written. This section models the fiber as the ground state of a statistical system and asks which aspects of the source remain.

11.1 Gibbs measures on the source space

Fix a target b, a distance d between byte strings that is $d(x, y) = 0$ only for $x = y$, and a graph G on S with edges representing single edits. For $s \in S$, define

$$u(s) = \#\{s' : s' \sim s \text{ in } G,\ C(s') \neq C(s)\},$$

as the number of neighbours that compile differently, and let $|s|$ denote the size of s. The energy is

$$E(s) = d(C(s), b) + \mu\, u(s) + \lambda|s|,$$

and the Gibbs measure at temperature $T > 0$ is $p_T(s) = Z_T^{-1} \exp(-E(s)/T)$.

The value $u(s)$ depends only on the neighbourhood of s, and

$$\sum_{s \in X} u(s) = |\partial X| \quad \text{for } X = C^{-1}(C(s)),$$

Thus, u is the pointwise version of the boundary count.

Remark 11.1 (Where the topology enters). p_T depends on G only through u. Without the term μu, the equilibrium measure is identical for every graph on S; the graph then influences only the dynamics. A Metropolis chain on a connected G with symmetric proposals has p_T as its stationary law, while G determines its mixing rate and where it becomes trapped. In what follows, the dynamics encodes the global topology and the energy encodes the local topology.

Remark 11.2 (Size). S is infinite because the dead store above can be repeated with any constant. Consequently, Z_T diverges unless there is a size term or a size cap. With $\lambda > 0$, the limit $T \to 0$ selects the shortest elements of the fiber, treating destroyed parts as absent, as in a minimum description length principle.

11.2 Soundness of the low temperature limit

Proposition 11.3. *Let S be finite, $C^{-1}(b) \neq \emptyset$, and define $\mu = \rho T$ and $\lambda = \ell T$ using constants $\rho, \ell \geq 0$. Then, as $T \to 0$, the measure p_T converges to*

$$p_0(s) \propto \mathbf{1}[C(s) = b]\, \exp(-\rho\, u(s) - \ell|s|),$$

whose support is the entire fiber. If instead $\mu > 0$ is fixed, p_T concentrates on the minimisers of $d(C(s), b) + \mu u(s)$. In general, these do not include every point in the fiber and may exclude all of them.

Proof. Under this scaling, $\exp(-E(s)/T) = \exp(-d(C(s), b)/T) \exp(-\rho u(s) - \ell|s|)$. The second factor is independent of T and is positive. The first is 1 on the fiber and no greater than $\exp(-\delta/T)$ elsewhere, where $\delta > 0$ is the smallest nonzero value of d on the finite set $C(S)$. If μ is fixed, the exponent $-E/T$ reaches its maximum precisely at the minimisers of E, and these points carry all the mass in the limit. □

11.3 What can be recovered

Write $s \approx s'$ if $C(s) = C(s')$. The destroyed part of s is its equivalence class in $S/\!\approx$, and its size up to n is quantified by

$$H_n(b) = \log_2 \left|C^{-1}(b) \cap \{s : |s| \leq n\}\right|.$$

The data processing inequality says that no function of b contains more information about s than b; therefore, the conditional entropy $H(s \mid b)$ is a hard lower bound. The terms μu and $\lambda|s|$ select a representative of the class. They serve as a prior and recover no information from the binary.

Erasure at -O3 is not invariably complete. Dead code that remains until the interprocedural passes is still included in size estimates and summaries, and can shift a marginal inlining decision in another function.

Definition 11.4. Consider $s \approx s'$ at the single-function level. The pair is *tier 1* if s and s' remain equivalent in every translation-unit context; it is *tier 2* if some context distinguishes them. Sources that compile differently on their own form a tier 3 pair.

The experiments below disprove the conjecture that tier 2 information affects the bytes only through D, and point to the following hypothesis.

Remark 11.5 (Conjecture). In a translation-unit context, $H(s \mid b_{\mathrm{TU}}) = H(s \mid b_f, a, D)$, where b_f denotes the bytes of the function itself and (a, D) its extended summary.

11.4 Global topology of the fiber

The fiber may divide into several components within G. At low temperature, a chain can move between components only by crossing an energy barrier Δ, which takes time of order $\exp(\Delta/T)$. The relevant invariant is the component structure of the sublevel sets $\{s : E(s) \le t\}$ as t changes—in other words, the persistence of H_0. A fiber component born at level 0 and merging at level Δ forms a finite bar $[0, \Delta)$. The chain's spectral gap and the Cheeger constant of G weighted by p_T bound the mixing time.

11.5 Experiments

All experiments were run with gcc 15.2 at -O3. They are small, exhaustive tests, not measurements on zstd.

Dead code fibers. Four base functions—a constant return, a loop sum, a call wrapper and a branch—were given up to two insertions selected from fifteen kinds of dead code, yielding 964 sources in total. Every source compiles to the bytes of its base function, so each base fiber contains 241 points and $H_2 = 7.91$ bits. Twelve of the fifteen kinds, among them dead loops, dead arrays and branches rendered dead by inlining a helper, leave no trace even in the interprocedural size summary and are therefore tier 1. The exceptions are branches guarded by a static variable that is never written. They retain their size in the summary because the compiler only discovers that the variable is never written during the interprocedural passes. They account for 1.69 of the 7.91 bits, and a caller close to the inlining threshold recovers between 1.62 and 1.68 of those bits. None are recovered for the constant base, which is always inlined.

Exact thermodynamics. The loop-sum base, with the fifteen dead kinds and five live edits and at most two insertions, produces 421 sources in 27 byte classes. Inserting, deleting or replacing one insertion defines G, whose maximum degree is 59; d is the byte edit distance.

- Proposition 11.3 is borne out. For the base target, whose fiber has 242 points, scaling $\mu = \rho T$ retains all 242 in the limit, whereas every tested fixed μ retains exactly one. For a second target, the fixed value $\mu = 10$ shifts the ground state entirely outside the fiber.

- The translation unit reveals more than D. With a caller at three inlining thresholds, $H(s \mid b_{\mathrm{TU}})$ is lower than $H(s \mid b_f, D)$ by between 0.26 and 0.83 bits. In one case, the function bytes are identical and both calls are inlined, yet the caller is 321 bytes long in one instance and 501 in the other. The dead guard makes the callee impure from the caller's perspective, preventing the caller from merging its two calls. This information is in a summary in a, not in a bit of D.

- The base fiber has two components in G: a large component with 241 points and an isolated point, giving one finite bar $[0, 33)$. At the isolated point, the live edit `a ^= 3` is applied twice. The edits cancel, amounting to algebraic erasure rather than dead code, and every neighbour of that point compiles differently. The chain's spectral gap drops from 7.6×10^{-2} at $T = 30$ to 5.4×10^{-7} at $T = 3$, and is numerically zero below $T = 1$. For the second target, the fiber is connected, has no finite bar, and the gap remains at 8.5×10^{-3} as $T \to 0$.

Recovery. Taking each of the 421 sources in turn as the hidden truth, the most probable element under the limit measure always compiles to the target bytes. It matches the hidden source in 6.4 percent of cases when only the function bytes are observed, and in 9.5 percent when the translation unit is observed. It preserves the same live edits in 91.4 and 92.9 percent of cases. The mean remaining entropy is 6.33 and 5.22 bits, respectively. An annealed Metropolis chain, run 1200 times from random starting points for 4000 steps each, ends at a source with the target bytes in 99.5 percent of runs.

These figures describe a toy space for one function and cannot, on their own, predict recovery from a real binary, where finding any candidate in the first place is the hard part.

12 Information inside a function

Section 10's theorem factors the fiber across functions through the summaries (a, D). Here we ask the same question within a single function: does one source edit alter only a small, local portion of the bytes, or does its effect spread throughout? The answer determines whether a function can be recovered piece by piece or needs to be read as a whole.

12.1 Rewrite groups

Let G be a group of behaviour-preserving rewrites of the binary, such as renaming registers while preserving the calling convention, or reordering independent instructions within a basic block. Byte equality is thus replaced by equality of G orbits. This gives us a ladder of targets: exact bytes, then the orbit of G, then agreement on tests, and finally equal behaviour. In general, only the first two are decidable; Rice's theorem rules out deciding the last, which is why byte equality remains our target.

Ladder test. We took the 87 functions of run r13 that compile, are not exact and have byte similarity at least 0.8. None of the 87 reaches any rung: not after removing nops, not under renaming that keeps the ABI, not under renaming together with reordering inside blocks, and not under unconstrained renaming. In 79 of them the instruction count differs, and in 65 it differs by exactly one instruction. The causes are type errors in the recovered source (a bool where the target returns an unsigned value, 32 against 64 bit operations, a pointer truncated in a test, a parameter typed as a pointer to a 16 byte value), not a different arrangement of the same instructions. So the near misses lie outside every orbit of G, and enlarging G would not help.

12.2 Blobs and atoms

Definition 12.1. For an edit e of the source, its *blob* $B(e)$ is the set of byte positions of the function that change when e is applied. For a family of edits E, two positions are in the same *atom* when they belong to exactly the same blobs $B(e)$, $e \in E$.

Once the family is large, the connected components of the union of the blobs merge into one component per function, making them uninformative. Atoms are the useful object: they are the coarsest partition of the bytes respected by every edit.

Small exact functions. Among the 111 byte exact functions of r13, with 2797 compiled edits, 72.6 percent of edits leave the bytes unchanged, and the atoms average 4.5 bytes.

Real source. Across five translation units of zstd (41 emitted functions, 1251 edits of real source, gcc 15.2 at -O3), 89 percent of edits leave the bytes unchanged, and a type change affects other functions in 38 of 109 cases. For edits that do change the bytes, the median changed share and the number of separate pieces per edit are: in bytes, 15 percent in 27 runs; in bytes with internal jump targets masked, 12 percent in 17 runs; in instructions, 21 percent in 7 runs; in instructions with registers abstracted, 13 percent in 8 runs; and in the dataflow graph with two rounds of Weisfeiler and Leman labels, 24 percent in 5 components. Depending on the coordinate, only between 12 and 26 percent of edit pairs have disjoint blobs. Only 7 percent of byte atoms lie on instruction boundaries, so the earlier impression that an atom corresponds to one instruction was just a coincidence of sizes.

Sequential test. In 26 functions, we fixed a prefix of source lines and varied the rest (337 line variants). Fixing the prefix eliminated 4 exact variants and added none. Among 365 pairs, we found no pair of non exact lines that became exact when combined. Thus, the fiber of a function is close to a product over its lines, even though the bytes affected by a single line are spread throughout the function.

Remark 12.2. Across functions, the fiber factors through summaries, as in Section 10. Within a function, a single edit's effect is spread across every coordinate we tested, much like a code word, while the set of correct sources remains nearly a product over lines. Recovery should therefore consider the whole function when proposing candidates, then check each line separately when accepting them.

13 Flags

As Section 11 shows, destroyed information cannot be recovered; what we can recover is the fiber $C^{-1}(b)$. The register machine preserves the program's dataflow, but not its meaning: a 64-bit value in a register could be a pointer, a size, a count or a truth value. A *flag* is a label that restores some of this meaning. Flags serve two purposes: they determine the source's form before any search begins, and they are checked for consistency across the call graph.

13.1 A hierarchy of levels

Recovery advances through levels, each fixed before the next and accepted only when the result still compiles to the target bytes.

1. Bytes: instructions and relocations, read directly from the binary.
2. Dataflow: values, calls and control flow, including clobbered registers, noreturn and tail calls.
3. Meaning: the type of each value, with flags for return kind, width, signedness, bool, pointer, array extent and structure field.
4. Effects: function summaries, namely purity, memory effects, constant parameters and clones. These are the summaries a of Section 10.
5. Structure: loops, branches and inlined callees, with flags for loop form, inlined call sites, unrolling and vectorisation.
6. Names: idioms such as an allocation wrapper, an error code, a size or a context pointer.
7. Prose: formatting and comments.

Levels 6 and 7 do not affect the bytes, so choices at those levels are free. The challenging levels are 3 to 5.

13.2 Flags as summary variables

Each flag is a variable on the call graph. Evidence from within the function and from call sites gives its value a likelihood, and belief propagation over the call graph yields a posterior. A flag with a confident posterior is fixed; an uncertain one becomes a search variable, which local search such as WalkSAT repairs if no candidate compiles exactly with its current value. Exact compilation and comparison remain the final test.

In a toy model of flag propagation, greedy retraction performed poorly, WalkSAT and Gibbs sampling tied, belief propagation was the most cautious, and combining the latter two brought no improvement. All solvers finished within about 2 percent of one another, indicating that the constraint is the available information, not the solver.

Return kinds on zstd. Suppose a function's return kind is one of void, integer, long or pointer. Among 609 zstd functions with source truth, the decompiler selects the correct kind for 83.7 percent. Incorporating call-site evidence (how callers use the returned register) raises accuracy to 87.7 percent: 32 functions are corrected and 8 are made incorrect. Belief propagation over forwarded returns yields no further improvement. The likelihood tables are set by hand and have not yet been fitted on held-out data.

13.3 Return width as a join

Let $W = \{1, 2, 4, 8\}$ be the lattice of register widths in bytes, ordered by size, with join max. For a function f let $\mathrm{def}(f) \in W$ be the width of the return register that f defines on every return path, counting an earlier zeroing of the wide register, and let $\mathrm{read}(c) \in W$ be the width of the return register that a caller reads after call site c.

Proposition 13.1. *Let w assign a width to every function subject to $w(f) \geq \mathrm{def}(f)$, $w(f) \geq \mathrm{read}(c)$ for every call site c of f, and $w(g) \geq w(f)$ whenever g returns the result of f unchanged. There is a least such assignment, obtained by starting from $w(f) = \max(\mathrm{def}(f), \max_c \mathrm{read}(c))$ and iterating the forwarding constraints.*

Proof. Each constraint has the form $w(g) \geq \phi(w)$ with ϕ monotone, and W is a finite lattice. The map raising every $w(g)$ to the join of its lower bounds is monotone on the finite product lattice W^F, so iteration from the starting point reaches its least fixed point by the Kleene theorem, after at most $3|F|$ steps. □

When the recovered prototype is narrower than this least width, callers read bits that the recovered callee never sets. In C, these show up as uninitialised variables, causing the program to take branches at random. Run r18 had 196 such sites; the largest group arose from the error test `ZSTD_isError`. The original implements this by clearing the whole register and then setting its lowest byte, but it had been recovered as returning bool. Seeding every prototype with its least width eliminated all 196 sites. The byte exact count increased from 145 to 156 functions in run r19, and all 11 newly exact functions had a widened return; no exact functions were lost. After a follow-up repair, the count reached 157 of 1139 in run r20.

Remark 13.2. Flags cannot restore the destroyed information $H(s \mid b)$; by data processing, nothing computed from b can. Instead, they correct errors in the parts of the source that the bytes do determine and, together with a readability prior, select which point of the fiber is printed.

14 Core logic

Earlier sections assess recovery against the strict standard $C_\theta(s) = b$. Here we document a weaker goal: each function's behaviour, and the tools developed to recover it. The motivation is practical: in zstd, one incorrect type costs one instruction and causes the function to count as failed, even if its logic is correct.

14.1 Logic is constant on the fiber

Let $[\![b]\!]$ denote the partial function computed by the bytes b of one function on the machine model, and $[\![s]\!]$ the partial function assigned by the C standard to a source s. Let $D(s)$ denote the set of inputs on which s has defined behaviour. A compiler C is correct if $[\![C(s)]\!](x) = [\![s]\!](x)$ for every $x \in D(s)$.

Proposition 14.1. *Suppose C is correct and $C(s) = C(s') = b$. Then $[\![s]\!]$ and $[\![s']\!]$ agree on $D(s) \cap D(s')$, and both agree there with $[\![b]\!]$.*

Proof. For $x \in D(s) \cap D(s')$, correctness gives $[\![s]\!](x) = [\![b]\!](x) = [\![s']\!](x)$. □

Thus, the logic is determined by the bytes alone; no flag is needed to express it. The fiber does not determine anything else, including names, types, or the text's form. The domains $D(s)$ may differ between two members of the fiber, so this statement does not extend to all inputs.

Definition 14.2. A source s recovers the core logic of b over a set of inputs X if, for some compiler setting θ', the compiled function $C_{\theta'}(s)$ returns the same value and leaves memory in the same state as b for every $x \in X$.

This is the third rung of the ladder from the section on information inside a function: exact bytes, the orbit of a rewrite group, tested behaviour, and then true behaviour. Rice's theorem makes true behaviour undecidable, so tested behaviour is the strongest target below exact bytes that can be checked.

14.2 Machine typing

Machine code contains no source types, only widths and operations. A typing that reflects this cannot conflict with the bytes.

Definition 14.3. A recovered function is machine typed if every variable has an unsigned integer type of width $w \in \{1, 2, 4, 8, 16\}$ bytes, every memory access reads or writes w bytes at an address of the form $p + k$, and every operation whose result depends on signedness (comparison, right shift, extension, division) names its signedness itself.

C unsigned arithmetic wraps modulo 2^{8w}, and byte access can alias any object, so each machine instruction has a machine typed statement with the same effect on the model state. This makes sign a property of operations rather than variables, in keeping with the lattice result below.

One choice, however, is not ours to make. A constant is an address constant if its value changes with the binary's load address. Since the rebuilt program has a different layout, an address constant must be represented as a symbol plus an offset, and the symbols must partition the image into objects.

Proposition 14.4. *Let P be the machine typed translation of a program, with every address constant written as symbol plus offset for a partition π of the image into objects. Suppose that in every execution of the original each pointer derived from an address in an object $o \in \pi$ stays inside o or one byte past its end, and that the program subtracts or orders two pointers only when they derive from the same object. Then P computes the same function under every layout that keeps the offsets inside each object.*

Proof. By induction on the execution. Non address values are equal in both runs because each statement mirrors its instruction. An address value is a pair of an object and an offset, and the offset is equal in both runs. Loads and stores inside an object see the same bytes at the same offsets. A subtraction or order comparison of two addresses has both in one object by hypothesis, so it depends only on the offsets. □

The hypothesis breaks down if π is too fine. In zstd, a pointer one past a string's end had been assigned its own object, losing the distance between a window base and the next source pointer; compression then followed a different path. Merging objects that touch fixes this.

Machine typing rules out incorrect types as a cause of incorrect behaviour. It does not rule out lowering errors, arguments lost to interprocedural register allocation, or code that was never C.

14.3 A lattice of flags

For every parameter slot and return, take the product $L = W \times K \times S$, where $W = \{0 < 1 < 2 < 4 < 8\}$ is the width chain, $K = \{\bot < \text{integer} < \text{pointer}\}$ the kind chain, and $S = \{\bot, \text{signed}, \text{unsigned}, \top\}$ is flat. Each use of a slot, within a function or at a call site, supplies a lower bound; forwarding a value between slots imposes an inequality. Since all constraints are monotone, the least fixed point exists, as it does for the return width in the section on flags. An assignment to a slot is sound if it lies below that slot's true flags, and tight if it equals them.

For zstd, which has 2423 parameter slots and 836 returns with source truth, the refined least fixed point yields the following results.

component	sound (percent)	tight (percent)
parameter width	100	93
parameter kind	100	76
parameter sign	97.9	39
return width	100	51
return kind	100	48

Width and kind are sound for every slot. Sign has 18 violations: when the range permits, the compiler emits a signed comparison for an unsigned value, or vice versa, so a use does not provide a bound on sign. Sign therefore belongs on the probabilistic side, along with names and comments. In 22 percent of functions, every slot is tight.

Supplying the true flags to the pipeline instead of the inferred flags raises the byte-exact count from 157 to 177 out of 1139. Flags account for only a small part of exactness; the rest comes from types within functions and source form.

14.4 Matching against a library

Fix the true summary a^*. The local fiber $L_j(a^*)$ from the section on translation unit fibers is nonempty, because the true source s_j^* belongs to it. Let $\sim_j$ be an equivalence relation on S_j, with moves that preserve both the bytes F_j and the summary σ_j for every value of the neighbour summaries.

Proposition 14.5. *If a library $\Lambda_j \subseteq S_j$ contains a member of the class of s_j^* under $\sim_j$, then $\Lambda_j \cap L_j(a^*)$ is not empty, and exact comparison finds a member of it.*

Proof. The class of s_j^* lies inside $L_j(a^*)$ because the moves preserve F_j and σ_j. Exact comparison of the bytes accepts every member of $L_j(a^*)$, so it accepts that one. □

Without the hypothesis on Λ_j, no conclusion is possible. The probabilistic version is this: let a proposer draw candidates from a distribution q_j on S_j, and let $m_j = q_j(L_j(a^*))$. If N draws are independent, the probability that at least one falls in the fiber is $1 - (1 - m_j)^N$, and the expected number of draws up to and including the first hit is $1/m_j$. A poor proposer costs extra draws but never compromises correctness, because acceptance relies on exact comparison. The independence of the draws is assumed, not proved.

For zstd, the library consisted of functions already recovered. When each function was left out in turn, transfer recovered 122 of the 128 functions at normal form distance 0 from another recovered function, and 0 of the 48 beyond that distance. Of the 961 unrecovered functions, only 9 were within distance 0.3 of a recovered function, and 2 became exact. Among the unrecovered functions, 360 form 91 groups of near copies with no recovered member. A library can therefore multiply seeds, but cannot create them.

14.5 A behaviour test

For a function f, choose a finite input set X_f: integers from a small set and buffers of 4096 pseudo random bytes for pointers. A debugger calls f in both the original and rebuilt programs on each $x \in X_f$, recording the return value at the width specified by the flags and the buffers afterward. Before comparison, values pointing into a buffer are replaced with the pair of buffer index and offset. The verdict is *differ* if, on any input where both calls return, their results differ, or if exactly one call faults; it is *agree* if at least three inputs return on both sides and all results match; otherwise, it is *inconclusive*.

Like the structural tools listed under Limitations, the test is one sided. For a deterministic function, a verdict of differ refutes the candidate and supplies a witness x. A verdict of agree establishes only that no difference was detected on X_f.

Two controls assess the test itself. Every byte-exact function must either agree or be inconclusive, since its bytes match the original. A function compared with a different function of the same signature should differ.

In zstd run r22, 862 functions were tested using 12 inputs apiece.

	agree	differ	inconclusive
byte exact (157)	144	0	13
not byte exact (705)	477	51	177

The first control passes without failures. In the second, 199 of 250 pairs differ; most of the pairs that agree are genuine near copies, such as the same routine in three legacy format versions. Comparing the original with itself identifies 9 functions as nondeterministic.

Not all agreements are equally strong. Of the 477, 329 agree on at least 8 inputs with at least 4 distinct memory results, while 52 produce a single result, mostly getters and early error returns. Random buffers rarely make a valid frame, so the codec's main path is seldom reached; only the return value and argument buffers are compared. The 329 provide evidence of correct logic, but not proof.

14.6 Repair with a gate

A verdict of differ supplies an input x on which the candidate and original disagree. This is a counterexample in the sense used in the section on counterexample-guided synthesis, and the loop works the same way: a proposer receives the disassembly, the candidate, and x, then returns a new candidate for the test to accept or reject. The proposer can be any procedure, including a language model, because it is never trusted. The same gate safeguards the final stage, where names, types, and comments are rewritten for readers: an edit is retained only if the function still passes.

The method therefore has three stages. Flags and summaries connect functions across the call graph. Within each local fiber, the decompiler, a library, or a proposer produces a candidate, which is checked either exactly or with the behaviour test. A final pass makes the text readable while retaining the gate.

15 Limitations

All results in these notes are proved with respect to the machine model Σ and a supplied, rather than inferred, ABI. Accordingly, box containment, the fibers of Section 4 and the loop invariants of Section 8 make no claims about faults, page permissions, concurrency or timing. They hold only if the binary actually uses the stated calling convention.

Recovery is judged by the strict standard $C_\theta(s) = b$. A candidate that computes the same function over the modelled domain is classified as semantic agreement, never as recovered source, since different register allocations and instruction selections routinely implement the same function. The reverse gap is equally important: if the search over Ω runs out of budget, the sole conclusion is that the candidates tested failed. This does not show that no source exists, either within the grammar or outside it.

The structural tools have the same one-sided character. Different stable 1-WL histograms prove that two graphs are not isomorphic, whereas matching histograms prove nothing, as C_6 and $2K_3$

show; moreover, graph isomorphism neither implies nor rules out semantic equality. Finally, a prototype fixed point certifies only that inference has stopped changing. Since no monotone order is known for Φ, convergence is enforced through budgets rather than guaranteed, and a stable set of prototypes does not prove that the recovered program is equivalent to the original.